

Programación Orientada a Objetos

La Programación Orientada a Objetos (POO) se basa en la agrupación de objetos de distintas clases que interactúan entre sí y que, en conjunto, consiguen que un programa cumpla su propósito. En Python cualquier elemento del lenguaje pertenece a una clase y todas las clases tienen el mismo rango y se utilizan del mismo modo.

Definición de clase, objeto, atributos y métodos

- Llamamos **clase** a la representación abstracta de un concepto. Por ejemplo, “perro”, “número entero” o “servidor web”.
- Las clases se componen de **atributos** y **métodos**.
- Un **objeto** es cada una de las instancias de una clase.
- Los **atributos** definen las características propias del objeto y modifican su estado. Son datos asociados a las clases y a los objetos creados a partir de ellas.
- Un **atributo de objeto** se define dentro de un método y pertenece a un objeto determinado de la clase instanciada.
- Los **métodos** son bloques de código (o funciones) de una clase que se utilizan para definir el comportamiento de los objetos.

Atributos de objetos

Para definir **atributos de objetos**, basta con definir una variable dentro de los métodos, es una buena idea definir todos los atributos de nuestras instancias en el **constructor**, de modo que se creen con algún valor válido.

Método constructor `__init__`

Como hemos visto anteriormente los atributos de objetos no se crean hasta que no hemos ejecutado el método. Tenemos un método especial, llamado **constructor** `__init__`, que nos permite inicializar los atributos de objetos. Este método se llama cada vez que se crea una nueva instancia de la clase (objeto).

Definiendo métodos. El parámetro `self`

El método **constructor**, al igual que todos los métodos de cualquier clase, recibe como primer parámetro a la instancia sobre la que está trabajando. Por convención a ese primer parámetro se lo suele llamar `self` (que podríamos traducir *como yo mismo*), pero puede llamarse de cualquier forma.

Para referirse a los **atributos de objetos** hay que hacerlo a partir del objeto `self`.

Definición de objetos

Vamos a crear una nueva clase:

```
import math
class punto():
    """ Representación de un punto en el plano, los atributos son x e y
    que representan los valores de las coordenadas cartesianas."""

    def __init__(self,x=0,y=0):
        self.x=x
        self.y=y

    def mostrar(self):
        return str(self.x)+":"+str(self.y)

    def distancia(self, otro):
        """ Devuelve la distancia entre ambos puntos. """
        dx = self.x - otro.x
        dy = self.y - otro.y
        return math.sqrt((dx*dx + dy*dy))
```

Para crear un objeto, utilizamos el nombre de la clase enviando como parámetro los valores que va a recibir el constructor.

```
>>> punto1=punto()
>>> punto2=punto(4,5)
>>> print(punto1.distancia(punto2))
6.4031242374328485
```

Podemos acceder y modificar los atributos de objeto:

```
>>> punto2.x
4
>>> punto2.x = 7
>>> punto2.x
7
```

Encapsulamiento en la programación orientada a objetos

En la unidad anterior terminamos viendo que teníamos la posibilidad de cambiar los valores de los atributos de un objeto. En muchas ocasiones es necesario que esta modificación no se haga directamente, sino que tengamos utilizar un método para realizar la modificación y poder controlar esa operación. También puede ser deseable que la devolución del valor de los atributos se haga utilizando un método. La característica de no acceder o modificar los valores de los atributos directamente y utilizar métodos para ello lo llamamos **encapsulamiento**.

Atributos privados

Las variables que comienzan por un doble guión bajo `__` la podemos considerar como **atributos privados**. Veamos un ejemplo:

```
>>> class Alumno():
...     def __init__(self,nombre=""):
...         self.nombre=nombre
...         self.__secreto="asdasd"
...
>>> a1=Alumno("jose")
>>> a1.__secreto
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
AttributeError: 'Alumno' object has no attribute '__secreto'
```

Propiedades: getters y setters

Para implementar la encapsulación y no permitir el acceso directo a los atributos, podemos poner los atributos ocultos y declarar métodos específicos para acceder y modificar los atributos.

- En Python, las **propiedades (getters)** nos permiten implementar la funcionalidad exponiendo estos métodos como atributos.
- Los métodos **setters** son métodos que nos permiten modificar los atributos a través de un método.

```
class circulo():
    def __init__(self,radio):
        self.radio=radio

    @property
    def radio(self):
        print("Estoy dando el radio")
        return self.__radio

    @radio.setter
    def radio(self,radio):
        if radio>=0:
            self.__radio = radio
        else:
            print("Radio debe ser positivo")
            self.__radio=0

>>> c1=circulo(3)
>>> c1.radio
Estoy dando el radio
3
>>> c1.radio=4
>>> c1.radio=-1
Radio debe ser positivo
>>> c1.radio
0
```

Herencia y delegación

Herencia

La herencia es un mecanismo de la programación orientada a objetos que sirve para crear clases nuevas a partir de clases preexistentes. Se toman (heredan) atributos y métodos de las clases bases y se los modifica para modelar una nueva situación.

La clase desde la que se hereda se llama clase base y la que se construye a partir de ella es una clase derivada.

Si nuestra clase base es la clase punto estudiadas en unidades anteriores, puedo crear una nueva clase de la siguiente manera:

```
class punto3d(punto):
    def __init__(self,x=0,y=0,z=0):
        super().__init__(x,y)
        self.z=z

    @property
    def z(self):
        print("Doy z")
        return self.__z

    @z.setter
    def z(self,z):
        print("Cambio z")
        self.__z=z

    def mostrar(self):
        return super().mostrar()+":"+str(self.__z)

    def distancia(self,otro):
        dx = self.__x - otro.__x
        dy = self.__y - otro.__y
        dz = self.__z - otro.__z
        return (dx*dx + dy*dy + dz*dz)**0.5
```

La clase punto3d hereda de la clase punto todos sus propiedades y sus métodos. En la clase hija hemos añadido la propiedad y el setter para el nuevo atributo z, y hemos modificado el constructor (sobrescritura) el método mostrar y el método distancia.

Creemos dos objetos de cada clase y veamos los atributos y métodos que tienen definido:

```
>>> p=punto(1,2)
>>> p3d=punto3d(1,2,3)
>>> p.distancia(punto(5,6))
5.656854249492381
>>> p3d.distancia(punto3d(2,3,4))
1.7320508075688772
```

La función super()

La función `super()` me proporciona una referencia a la clase base. Como vemos en ejemplo hemos reescrito algunos métodos: `__init__`, `mostrar()` y `distancia()`. En algunos de ellos es necesario usar el método de la clase base. Para acceder a esos métodos usamos la función `super()`.

```
...
def __init__(self,x=0,y=0,z=0):
    super().__init__(x,y)
    self.z=z
...
def mostrar(self):
    return super().mostrar()+":"+str(self.__z)
...
```

Delegación

Llamamos delegación a la situación en la que una clase contiene (como atributos) una o más instancias de otra clase, a las que delegará parte de sus funcionalidades.

A partir de la clase `punto`, podemos crear la clase `circulo` de esta forma:

```
class circulo():

    def __init__(self,centro,radio):
        self.centro=centro
        self.radio=radio

    def mostrar(self):
        return"Centro:{0}-
Radio:{1}".format(self.centro.mostrar(),self.radio)
```

Y creamos un objeto `circulo`:

```
>>> c1=circulo(punto(2,3),5)
>>> print(c1.mostrar())
Centro:2:3-Radio:5
```